

CSE232 시스템프로그래밍(캡스톤디자인)

5주차 첫 번째 수업 · 보조 자료

실습 코드 한 줄씩 읽기

경민기

이 교안을 읽는 방법

왼쪽은 labs/class05-01의 소스를 행 번호와 함께 그대로 옮긴 것입니다.

파란 글자 설명은 그 화면 행들의 요점입니다. 코드와 색으로 구분됩니다.

발표자 노트와 Markdown 원고에 매 줄의 설명이 있습니다.

챕터는 교안 본편과 같습니다: 1부 시스템 식별, 2부 sysconf, 3부 자원 한계, 4부 /proc.
sysprobe.c는 1부(main의 uname 블록)와 2부(probe 함수)에서 함께 씁니다.

1부

시스템 식별

uname과 sysinfo로 실행 환경을 확인합니다 · sysprobe.c main 블록,
sysinfo_demo.c 전체

sysprobe.c · 25-40행 · main 진입과 uname

```
25 int main(int argc, char **argv)
26 {
27     if (argc != 1) {
28         fprintf(stderr, "usage: %s\n", argv[0]);
29         return 2;
30     }
31     struct utsname u;
32     if (uname(&u) == -1) {
33         perror("uname");
34         return 1;
35     }
36     printf("sysname=%s\n", u.sysname);
37     printf("nodename=%s\n", u.nodename);
38     printf("release=%s\n", u.release);
39     printf("version=%s\n", u.version);
40     printf("machine=%s\n", u.machine);
```

25-30행 · main 진입

인자 없는 프로그램: 인자가 있으면 usage와 종료 2.

31-35행 · uname 호출

구조체를 채우게 하고 실패를 perror와 종료 1로 보고한다.

36-40행 · 식별 출력

sysname·nodename·release를 차례로 출력하기 시작한다.

sysinfo_demo.c · 1-16행

```
01  /* sysinfo(2)는 Linux 전용 인터페이스다.
02     이 파일에서는 조건부 컴파일로 지원 환경을 구분한다. */
03  #if defined(__linux__)
04
05  #include <errno.h>
06  #include <stdio.h>
07  #include <sys/sysinfo.h>
08
09  int main(void)
10  {
11      struct sysinfo si;
12      if (sysinfo(&si) == -1) {
13          perror("sysinfo");
14          return 1;
15      }
16      printf("uptime=%ld sec\n", si.uptime);
```

1-2행 · 파일 주석

Linux 전용임을 먼저 밝히고
조건부 컴파일을 예고한다.

3행 · #if

__linux__일 때만 이 아래
블록을 컴파일한다.

11-16행 · 조회와 uptime

sysinfo가 구조체를 채우고
성공 뒤 가동 시간을 출력한다.

sysinfo_demo.c · 17-32행

```
17  /* loads는 65536을 1로 하는 고정소수점 값이다. */
18  printf("loadavg=%.2f %.2f %.2f\n",
19         (double)si.loads[0] / (1 << SI_LOAD_SHIFT),
20         (double)si.loads[1] / (1 << SI_LOAD_SHIFT),
21         (double)si.loads[2] / (1 << SI_LOAD_SHIFT));
22  printf("procs=%u\n", si.procs);
23  /* 램 크기는 필드 값에 mem_unit을 곱해 바이트로 환산한다. */
24  printf("totalram=%lu bytes\n", si.totalram * (unsigned
↳ long)si.mem_unit);
25  printf("freeram=%lu bytes\n", si.freeram * (unsigned
↳ long)si.mem_unit);
26  printf("mem_unit=%u\n", si.mem_unit);
27  if (fflush(stdout) == EOF) {
28      perror("stdout");
29      return 1;
30  }
31  return 0;
32 }
```

17-21행 · loadavg

loads는 65536이 1인
고정소수점이다. 나눠서 환산한다.

22-26행 · 램과 단위

필드 값에 mem_unit을 곱해
바이트로 만들어 출력한다.

27-32행 · 종료

flush 뒤 0을 반환해 Linux
블록을 마친다.

sysinfo_demo.c · 33-44행 · #else 분기

```
33
34 #else /* not __linux__ */
35
36 #include <stdio.h>
37
38 int main(void)
39 {
40     puts("sysinfo(2) is Linux-only: not available on this system");
41     return 1;
42 }
43
44 #endif
```

33-36행 · #else 분기

Linux가 아니면 stdio만
넣고 다른 main을 컴파일한다.

38-42행 · 안내와 실패

전용 인터페이스가 없음을 알리고
종료 상태 1로 끝낸다.

43-44행 · #endif

조건부 컴파일의 끝. 두 main
중 하나만 존재한다.

2부

sysconf와 경계값

실행 시점에 경계값을 조회합니다 · sysprobe.c의 probe 함수와 sysconf 호출

sysprobe.c · 1-12행 · probe 함수

```
01 #include <errno.h>
02 #include <inttypes.h>
03 #include <stdio.h>
04 #include <sys/utsname.h>
05 #include <unistd.h>
06
07 /* sysconf는 실패해도 errno를 바꾸지 않을 수 있으므로
08    호출 직전에 0으로 만들고 -1의 의미를 나누어 본다. */
09 static int probe(const char *label, int name)
10 {
11     errno = 0;
12     long value = sysconf(name);
```

1-5행 · 헤더

errno, printf, uname,
sysconf 선언을 얻는다.

7-8행 · 계약 주석

sysconf는 실패해도 errno를
안 바꿀 수 있음을 적어 두었다.

9-12행 · probe 시작

호출 직전 errno=0, 그리고
sysconf 호출로 long을 받는다.

sysprobe.c · 13-23행 · probe 함수

```
13     if (value == -1) {  
14         if (errno != 0) {  
15             perror(label);  
16             return 1;  
17         }  
18         printf("%s=unlimited\n", label);  
19         return 0;  
20     }  
21     printf("%s=%ld\n", label, value);  
22     return 0;  
23 }
```

13-17행 · 실패 갈래

-1에 errno가 남아 있으면
perror로 보고하고 1을 반환한다.

18-19행 · 무한 갈래

errno가 0이면 unlimited를
찍고 실패로 세지 않는다.

21-23행 · 성공 경로

값을 %ld로 출력하고 0을
반환해 probe를 마친다.

sysprobe.c · 41-52행 · sysconf 호출과 종료

```
41     int status = 0;
42     status |= probe("cpus_online", _SC_NPROCESSORS_ONLN);
43     status |= probe("pagesize", _SC_PAGESIZE);
44     status |= probe("clk_tck", _SC_CLK_TCK);
45     status |= probe("open_max", _SC_OPEN_MAX);
46     status |= probe("arg_max", _SC_ARG_MAX);
47     if (fflush(stdout) == EOF) {
48         perror("stdout");
49         return 1;
50     }
51     return status;
52 }
```

41-46행 · sysconf 다섯 값

같은 probe 함수로 조회하고
실패 비트를 status에 모은다.

47-50행 · 출력 flush

stdout 버퍼를 확실히 비우고
파이프 끊김까지 검사한다.

51-52행 · 종료

모은 status가 곧 프로그램의
종료 상태가 된다.

3부

자원 한계

getrlimit과 setrlimit으로 프로세스의 상한을 다룹니다 · limit_show.c,
nofile_demo.c

limit_show.c · 1-16행

```
01 #include <inttypes.h>
02 #include <stdio.h>
03 #include <sys/resource.h>
04
05 struct limit_entry {
06     const char *name;
07     int resource;
08 };
09
10 static int show(const struct limit_entry *entry)
11 {
12     struct rlimit rl;
13     if (getrlimit(entry->resource, &rl) == -1) {
14         perror(entry->name);
15         return 1;
16     }
```

1-3행 · 헤더

inttypes와 stdio, 그리고 struct rlimit 선언을 얻는다.

5-8행 · 자료 구조

자원의 이름과 RLIMIT 상수를 짝지은 작은 구조체다.

12-16행 · getrlimit

자원을 조회해 구조체를 채우고 실패를 검사한다.

limit_show.c · 17-28행

```
17     printf("%-7s", entry->name);
18     if (rl.rlim_cur == RLIM_INFINITY)
19         printf(" soft=unlimited");
20     else
21         printf(" soft=%ju", (uintmax_t)rl.rlim_cur);
22     if (rl.rlim_max == RLIM_INFINITY)
23         printf(" hard=unlimited\n");
24     else
25         printf(" hard=%ju\n", (uintmax_t)rl.rlim_max);
26     return 0;
27 }
28
```

17행 · 이름 출력

%-7s로 이름 칸을 고정폭으로 맞춘다.

18-21행 · soft

RLIM_INFINITY면 unlimited, 아니면 %ju로 숫자를 찍는다.

22-26행 · hard와 반환

같은 방식으로 hard를 출력하고 줄을 마쳐 0을 반환한다.

limit_show.c · 29-48행 · main과 자원 표

```
29 int main(void)
30 {
31     static const struct limit_entry table[] = {
32         {"nofile", RLIMIT_NOFILE},
33         {"as",      RLIMIT_AS},
34         {"stack",   RLIMIT_STACK},
35         {"fsize",   RLIMIT_FSIZE},
36         {"cpu",     RLIMIT_CPU},
37         {"nproc",   RLIMIT_NPROC},
38         {"core",    RLIMIT_CORE},
39     };
40     int status = 0;
41     for (size_t i = 0; i < sizeof table / sizeof table[0]; ++i)
42         status |= show(&table[i]);
43     if (fflush(stdout) == EOF) {
44         perror("stdout");
45         return 1;
46     }
47     return status;
48 }
```

31-39행 · 자원 표

일곱 종류의 RLIMIT 이름과 상수를 짝지어 둔다.

40-42행 · 반복 조회

sizeof로 항목 수를 계산해 show를 각각 호출한다.

43-48행 · 종료

flush 검사 뒤 모든 status를 종료 상태로 돌려준다.

nofile_demo.c · 1-15행

```
01 #include <errno.h>
02 #include <fcntl.h>
03 #include <inttypes.h>
04 #include <stdio.h>
05 #include <stdlib.h>
06 #include <string.h>
07 #include <sys/resource.h>
08 #include <unistd.h>
09
10 static void print_limit(const char *label)
11 {
12     struct rlimit rl;
13     if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
14         perror("getrlimit");
15         exit(1);
16     }
17 }
```

1-8행 · 헤더

open, malloc, strcmp, getrlimit 등에 필요한 선언.

10-13행 · print_limit

FD 한계를 조회해 구조체에 채우는 작은 헬퍼다.

14-15행 · 실패 처리

조사 헬퍼는 exit(1)로 바로 끝낸다: 이후 진행이 무의미하다.

nofile_demo.c · 16-34행

```
16     }
17     if (rl.rlim_cur == RLIM_INFINITY)
18         printf("%s soft=unlimited", label);
19     else
20         printf("%s soft=%ju", label, (uintmax_t)rl.rlim_cur);
21     if (rl.rlim_max == RLIM_INFINITY)
22         printf(" hard=unlimited\n");
23     else
24         printf(" hard=%ju\n", (uintmax_t)rl.rlim_max);
25 }
26
27 int main(int argc, char **argv)
28 {
29     int raise_requested = argc == 3 && strcmp(argv[1], "--raise")
30     ↪ == 0;
31     if (!(argc == 1 || raise_requested)) {
32         fprintf(stderr, "usage: %s [--raise N]\n", argv[0]);
33         return 2;
34     }
35     print_limit("before");
```

16-24행 · 한계 출력

soft·hard 각각 무한 여부를
가려 한 줄로 찍는다.

27-33행 · 인자 검사

인자 없음 또는 --raise N만
허용하고, 아니면 usage와 2.

34행 · before

실험 전 현재 한계를 먼저
기록해 둔다.

nofile_demo.c · 35-52행 · --raise 인자 처리

```
35
36     if (raise_requested) {
37         char *end = NULL;
38         errno = 0;
39         unsigned long long want = strtoull(argv[2], &end, 10);
40         if (end == argv[2] || *end != '\0' || errno != 0) {
41             fprintf(stderr, "invalid number: %s\n", argv[2]);
42             return 2;
43         }
44         struct rlimit rl;
45         if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
46             perror("getrlimit");
47             return 1;
48         }
49         /* 일반 사용자는 hard까지 상향할 수 있다. */
50         rlim_t target = (rlim_t)want;
51         if (rl.rlim_max != RLIM_INFINITY && target > rl.rlim_max)
52             target = rl.rlim_max;
```

37-43행 · 숫자 파싱

strtoull로 N을 읽고 끝
포인터와 errno로 검증한다.

44-48행 · 현재 한계

상향 전에 getrlimit로 hard를
확인한다.

49-52행 · 상한 계산

요청이 hard를 넘으면 hard로
낮춘다: 일반 사용자의 규칙.

nofile_demo.c · 53-72행 · 한계 변경과 측정 준비

```
53     struct rlimit next = { target, rl.rlim_max };
54     if (setrlimit(RLIMIT_NOFILE, &next) == -1) {
55         perror("setrlimit");
56         return 1;
57     }
58     print_limit("after ");
59 }
60
61 struct rlimit rl;
62 if (getrlimit(RLIMIT_NOFILE, &rl) == -1) {
63     perror("getrlimit");
64     return 1;
65 }
66 rlim_t cap = rl.rlim_cur == RLIM_INFINITY ? 65536 : rl.rlim_cur;
67 int *fds = malloc(sizeof *fds * (size_t)cap);
68 if (fds == NULL) {
69     perror("malloc");
70     return 1;
71 }
72 int opened = 0;
```

53-58행 · setrlimit

새 soft와 기존 hard로
구조체를 만들어 적용한다.

58행 · after

변경 직후 한계를 다시 출력해
실제 값을 확인한다.

61-67행 · 측정 준비

soft를 다시 읽고 배열 크기와
버퍼를 마련한다.

nofile_demo.c · 73-90행 · FD 열기 반복

```
73     while (opened < (int)cap) {
74         int fd = open("/dev/null", O_RDONLY);
75         if (fd == -1) {
76             if (errno == EMFILE)
77                 break;
78             perror("open");
79             while (opened > 0)
80                 close(fds[--opened]);
81             free(fds);
82             return 1;
83         }
84         fds[opened++] = fd;
85     }
86     printf("opened=%d stopped=", opened);
87     if (opened == (int)cap)
88         printf("cap\n");
89     else
90         printf("EMFILE\n");
```

73-74행 · 반복과 open

배열이 찰 때까지 /dev/null을 계속 연다.

75-82행 · EMFILE과 정리

EMFILE이면 실험 성공, 다른 오류면 다 닫고 실패 종료다.

84-90행 · 결과

연 FD를 저장하고 opened와 멈춘 이유를 출력한다.

nofile_demo.c · 91-106행 · 정리와 종료

```
91     int close_failed = 0;
92     while (opened > 0)
93         if (close(fds[--opened]) == -1)
94             close_failed = 1;
95     free(fds);
96     if (close_failed) {
97         perror("close");
98         return 1;
99     }
100     printf("closed all descriptors\n");
101     if (fflush(stdout) == EOF) {
102         perror("stdout");
103         return 1;
104     }
105     return 0;
106 }
```

91-95행 · 자원 반환

연 순서의 반대로 모두 닫고
배열을 해제한다.

96-99행 · close 검사

닫기 실패까지 놓치지 않고
오류로 보고한다.

100-106행 · 종료

완료 문구와 flush 검사 뒤
0으로 끝낸다.

4부

/proc 관찰 창

표준 API 바깥에서 Linux의 상태를 관찰합니다 · proc_probe.c 전체

proc_probe.c · 1-20행 · filter_lines

```
01 #include <dirent.h>
02 #include <errno.h>
03 #include <stdio.h>
04 #include <string.h>
05
06 /* keys로 시작하는 줄만 화면에 남긴다. 실패는 -1, 성공은 찾은 줄 수. */
07 static int filter_lines(const char *path, const char *const
    ↳ keys[], size_t nkeys)
08 {
09     FILE *f = fopen(path, "r");
10     if (f == NULL) {
11         fprintf(stderr, "%s: %s\n", path, strerror(errno));
12         return -1;
13     }
14     char line[256];
15     int found = 0;
16     while (fgets(line, sizeof line, f) != NULL) {
17         for (size_t i = 0; i < nkeys; ++i) {
18             if (strncmp(line, keys[i], strlen(keys[i])) == 0) {
19                 fputs(line, stdout);
20                 ++found;
            }
        }
    }
```

1-4행 · 헤더

opendir·readdir, errno,
stdio, strncmp 선언.

6-7행 · 필터 설계

keys 접두어로 줄을 골라
찾은 줄 수를 돌려준다.

9-20행 · 열기와 반복

fopen 오류를 strerror로
알리고 fgets로 한 줄씩 읽는다.

proc_probe.c · 21-34행 · filter_lines

```
21         }
22     }
23 }
24 if (ferror(f)) {
25     fprintf(stderr, "%s: read error\n", path);
26     fclose(f);
27     return -1;
28 }
29 if (fclose(f) == EOF) {
30     fprintf(stderr, "%s: %s\n", path, strerror(errno));
31     return -1;
32 }
33 return found;
34 }
```

21-23행 · 반복 닫음

키 반복과 줄 반복의
중괄호를 정리한다.

24-28행 · 읽기 오류

ferror로 fgets 실패를
EOF와 구분해 검사한다.

29-34행 · 닫기 검사

fclose 실패까지 확인하고
찾은 줄 수를 반환한다.

proc_probe.c · 35-55행 · first_line

```
35
36 static int first_line(const char *path)
37 {
38     FILE *f = fopen(path, "r");
39     if (f == NULL) {
40         fprintf(stderr, "%s: %s\n", path, strerror(errno));
41         return -1;
42     }
43     char line[256];
44     if (fgets(line, sizeof line, f) == NULL) {
45         fprintf(stderr, "%s: empty\n", path);
46         fclose(f);
47         return -1;
48     }
49     fputs(line, stdout);
50     if (fclose(f) == EOF) {
51         fprintf(stderr, "%s: %s\n", path, strerror(errno));
52         return -1;
53     }
54     return 0;
55 }
```

36-42행 · 첫 줄 열기

fopen 오류를 같은 형식으로 알린다.

43-48행 · 빈 파일

fgets가 바로 NULL이면 내용이 없는 것이다.

49-55행 · 출력과 반환

첫 줄만 찍고 닫기 검사 뒤 0을 반환한다.

proc_probe.c · 56–76행 · count_fd

```
56
57 /* readdir의 NULL은 끝과 오류를 함께 표현하므로 errno로 나눈다. */
58 static int count_fd(void)
59 {
60     DIR *dir = opendir("/proc/self/fd");
61     if (dir == NULL) {
62         fprintf(stderr, "/proc/self/fd: %s\n", strerror(errno));
63         return -1;
64     }
65     int count = 0;
66     for (;;) {
67         errno = 0;
68         struct dirent *ent = readdir(dir);
69         if (ent == NULL) {
70             if (errno != 0) {
71                 fprintf(stderr, "readdir: %s\n", strerror(errno));
72                 closedir(dir);
73                 return -1;
74             }
75             break;
76     }
```

57–60행 · opendir

/proc/self/fd를 디렉토리 스트림으로 연다.

65–68행 · errno=0

readdir 직전에 errno를 0으로 초기화한다.

69–76행 · NULL 판별

NULL은 끝 또는 오류: errno로 둘로 나눈다.

proc_probe.c · 77-85행 · count_fd

```
77         if (strcmp(ent->d_name, ".") != 0 && strcmp(ent->d_name,  
78             ↳ "..") != 0)  
79             ++count;  
80     }  
81     if (closedir(dir) == -1) {  
82         perror("closedir");  
83         return -1;  
84     }  
85     return count;  
86 }
```

77-78행 · 개수 세기

.과 ..만 제외한 이름마다
count를 늘린다.

80-83행 · 닫기 검사

closedir 실패도 반환값으로
확인한다.

84행 · 반환

open 중이던 스트림 자체도
항목이었다는 점을 기억한다.

proc_probe.c · 86–104행 · main

```
86
87 int main(void)
88 {
89     static const char *const status_keys[] = {"VmRSS:", "Threads:"};
90     int status = 0;
91     printf("/proc/self/status\n");
92     if (filter_lines("/proc/self/status", status_keys,
93                     sizeof status_keys / sizeof status_keys[0]) <
94         0)
95         status = 1;
96     printf("/proc/loadavg\n");
97     if (first_line("/proc/loadavg") == -1)
98         status = 1;
99     printf("/proc/self/fd\n");
100    int count = count_fd();
101    if (count < 0) {
102        status = 1;
103    } else {
104        printf("count=%d\n", count);
105    }
```

89행 · 필터 키

status에서 VmRSS와
Threads 줄만 골라낸다.

90–97행 · 관찰 3종

status 필터, loadavg 첫 줄,
각 실패를 status에 모은다.

98–104행 · fd 세기

count_fd 결과도 같은 방식으로
성공 여부에 반영한다.

proc_probe.c · 105–110행 · main

```
105     if (fflush(stdout) == EOF) {  
106         perror("stdout");  
107         return 1;  
108     }  
109     return status;  
110 }
```

105–108행 · flush

stdout을 확정하고 출력
실패를 검사한다.

109–110행 · 종료

모든 status를 반환한다:
하나라도 실패하면 1이다.

관찰 요점

같은 파일이 명령마다 다른
값을 주는 이유를 설명한다.

코드 리딩을 마치며

다섯 예제 모두 조회 성공 뒤에만 결과를 사용했습니다.

sysconf와 readdir는 호출 직전 errno=0으로 초기화하는 같은 계약을 공유합니다.

rlim_t와 long처럼 자료형은 출력 변환으로 명확히 했습니다.

실패 경로에서도 연 자원은 모두 닫았습니다.

챕터별 매 줄 설명은 발표자 노트와 Markdown 원고에 함께 있습니다.

함수는 어디에 사는가 · 원형과 구현

세 층: 원형은 헤더, 사용자 공간 구현은 glibc, 실제 일은 커널이 합니다.

시스템 호출 래퍼: `uname`, `sysinfo`, `getrlimit`·`setrlimit`, `open`·`close`.

조합형 함수: `sysconf`, `opendir`·`readdir`, `stdio`, `malloc`이 여기 있습니다.

순수 사용자 공간: `strtoull`, `strcmp` 계열, `exit`은 커널을 호출하지 않습니다.

원형의 위치는 man 페이지 SYNOPSIS의 `#include` 줄에서 바로 확인합니다.

확인: `nm -D libc.so.6 · strace`

커널 구현 위치 · kernel.org

kernel/sys.c: newuname, sysinfo, do_prlimit, sethostname.

fs/open.c: openat과 close. fs/readdir.c: getdents64.

mm/mmap.c: brk와 mmap. malloc이 커널에서 받아 오는 통로입니다.

include/linux/utsname.h와 init/version.c: new_utsname과 init_uts_ns.

링크는 kernel.org stable 저장소의 linux-7.2.y 브랜치로 연결합니다.

시간 함수의 위치 · kernel.org

달력 시각: `time`, `localtime` · `gmtime`, `mktime`, `strftime`, `difftime`.

경과 시간: `clock_gettime`, `clock_getres` — 원형은 모두 `<time.h>`입니다.

`time(2)`: glibc가 `clock_gettime(CLOCK_REALTIME)` 호출로 구현합니다.

`clock_gettime(2)`: vDSO에서 바로 값을 돌려주고, 없으면 시스템 호출로 갑니다.

`localtime` · `strftime(3)`: 시간대 해석과 문자열 변환은 glibc 안에서 일어납니다.